

GitHub-Ready authController.js

The JWT secret has been moved fully to the environment variable JWT_SECRET. The hard-coded fallback secret from the uploaded file should not be committed to GitHub.

```
const jwt = require('jsonwebtoken');
const User = require('../models/User');

// Helper to generate JWT Token
const generateToken = (id) => {
  if (!process.env.JWT_SECRET) {
    throw new Error('JWT_SECRET is not defined in environment variables');
  }

  return jwt.sign({ id }, process.env.JWT_SECRET, {
    expiresIn: '30d',
  });
};

const registerUser = async (req, res) => {
  try {
    const { name, email, password } = req.body;

    // Validate inputs
    if (!name || !email || !password) {
      return res.status(400).json({ success: false, message: 'Please add all fields' });
    }

    if (password.length < 6) {
      return res.status(400).json({ success: false, message: 'Password must be at least 6 characters' });
    }

    // Check if user already exists
    const userExists = await User.findOne({ email });
    if (userExists) {
      return res.status(400).json({ success: false, message: 'User already exists' });
    }

    // Create user (password hashing is done in pre-save hook)
    const user = await User.create({
      name,
      email,
      password,
    });

    if (user) {
      return res.status(201).json({
        success: true,
        data: {
          _id: user._id,
          name: user.name,
          email: user.email,
          token: generateToken(user._id),
        }
      });
    } else {
      return res.status(400).json({ success: false, message: 'Invalid user data' });
    }
  } catch (error) {
    console.error('Registration error:', error.message);
    return res.status(500).json({ success: false, message: 'Server error during registration' });
  }
};

const loginUser = async (req, res) => {
  try {
    const { email, password } = req.body;

    // Validate inputs
    if (!email || !password) {
      return res.status(400).json({ success: false, message: 'Please provide email and password' });
    }
  }
};
```

```

// Check if user exists
const user = await User.findOne({ email });
if (!user) {
  return res.status(401).json({ success: false, message: 'Invalid credentials' });
}

// Check password
const isMatch = await user.comparePassword(password);
if (!isMatch) {
  return res.status(401).json({ success: false, message: 'Invalid credentials' });
}

return res.json({
  success: true,
  data: {
    _id: user._id,
    name: user.name,
    email: user.email,
    token: generateToken(user._id),
  }
});
} catch (error) {
  console.error('Login error:', error.message);
  return res.status(500).json({ success: false, message: 'Server error during login' });
}
};

const getUserProfile = async (req, res) => {
  try {
    // req.user is set by protect middleware
    return res.json({
      success: true,
      data: {
        _id: req.user._id,
        name: req.user.name,
        email: req.user.email,
        createdAt: req.user.createdAt,
      }
    });
  } catch (error) {
    console.error('Profile retrieval error:', error.message);
    return res.status(500).json({ success: false, message: 'Server error fetching user profile' });
  }
};

module.exports = {
  registerUser,
  loginUser,
  getUserProfile,
};

```

.env example (do not upload this file)

```
JWT_SECRET=your_strong_random_secret_here
```

.gitignore: add **.env** so your secret is not uploaded to GitHub.